UNIT-02 DATA STRUCTURES AND SEQUENCES

1Q) Write about strings and its characteristics.

Ans) 🔷 Strings in Python

✅ Definition

A string is a sequence of characters enclosed in quotes.

👉 Quotes can be:

- Single ' '
- Double " "
- Triple ''' ''' or """ """

📌 Examples

a = 'Hello'
b = "Python"
c = '''This is
multi-line string'''

🔷 Characteristics of Strings

1️⃣ Strings are Immutable

👉 Once created, they cannot be changed

Example

text = "hello"

```
text[0] = 'H'   # ❌ Error
```

✔ Correct way:

```
text = "hello"
text = "H" + text[1:]
print(text)   # Hello
```

2️⃣ Strings are Indexed

👉 Each character has a position (index)

- Starts from 0
- Negative indexing is also allowed

Example

```
text = "Python"

print(text[0])   # P
print(text[3])   # h
print(text[-1])  # n
```

3️⃣ Strings are Ordered

👉 Characters maintain order

```
text = "ABC"
# Order: A → B → C (fixed)
```

4️⃣ Strings support Slicing

👉 Extract a part of string

Syntax:

string[start : end : step]

Example

text = "Python"

```
print(text[0:3])   # Pyt
print(text[2:])    # thon
print(text[:4])    # Pyth
print(text[::-1])  # nohtyP (reverse)
```

5️⃣ Strings are Iterable

👉 Can loop through characters

```
text = "Hi"

for ch in text:
  print(ch)
```

Output:

```
H
i
```

6️⃣ Strings support Concatenation

👉 Joining strings using +

```
a = "Hello"
b = "World"

print(a + " " + b)   # Hello World
```

7️⃣ Strings support Repetition

👉 Using *

```
text = "Hi"

print(text * 3)   # HiHiHi
```

8️⃣ Strings are Case Sensitive

👉 Uppercase and lowercase are different

```
print("hello" == "Hello")   # False
```

9️⃣ Strings can contain Any Characters

👉 Letters, numbers, symbols, spaces

```
text = "Akhil123!@#"
```

🔟 Strings are Stored as Sequence of Unicode Characters

👉 Python supports all languages

text = "नमस्ते"

2Q) write String functions.

Ans) 🔷 String Functions in Python

- 1. lower()

👉 Converts string to lowercase

```
text = "HELLO"
print(text.lower())   # hello
```

- 2. upper()

👉 Converts string to uppercase

```
text = "hello"
print(text.upper())   # HELLO
```

- 3. title()

👉 Converts first letter of each word to uppercase

```
text = "hello world"
print(text.title())   # Hello World
```

🔹 4. capitalize()

👉 Capitalizes only first letter

```
text = "python"
print(text.capitalize())   # Python
```

🔹 5. strip()

👉 Removes spaces from both sides

```
text = "  hello  "
print(text.strip())   # hello
```

🔹 6. lstrip() and rstrip()

👉 Remove spaces from left/right

```
text = "  hello  "

print(text.lstrip())  # "hello  "
print(text.rstrip())  # "  hello"
```

🔹 7. replace()

👉 Replaces part of string

```
text = "hello"
print(text.replace("h", "H"))   # Hello
```

🔹 8. find()

👉 Returns index of first occurrence
👉 Returns -1 if not found

```
text = "banana"
print(text.find("a"))   # 1
```

🔹 9. index()

👉 Same as find() but gives error if not found

```
text = "banana"
print(text.index("n"))   # 2
```

🔹 10. count()

👉 Counts occurrences

```
text = "banana"
print(text.count("a"))   # 3
```

🔹 11. startswith()

👉 Checks beginning of string

```
text = "Python"
print(text.startswith("Py"))   # True
```

🔹 12. endswith()

👉 Checks end of string

```
text = "Python"
print(text.endswith("on"))   # True
```

🔹 13. split()

👉 Splits string into list

```
text = "apple,banana,grape"
print(text.split(","))   # ['apple', 'banana', 'grape']
```

🔹 14. join()

👉 Joins list into string

```
words = ["Hello", "World"]
print(" ".join(words))   # Hello World
```

🔹 15. isalpha()

👉 Checks only letters

```
text = "Hello"
print(text.isalpha())   # True
```

🔹 16. isdigit()

👉 Checks only numbers

```
text = "123"
print(text.isdigit())   # True
```

🔹 17. isalnum()

👉 Checks letters + numbers

```
text = "abc123"
print(text.isalnum())   # True
```

🔹 18. isspace()

👉 Checks only spaces

```
text = "   "
print(text.isspace())   # True
```

🔹 19. swapcase()

👉 Changes uppercase to lowercase and vice versa

```
text = "PyThOn"
print(text.swapcase())   # pYtHoN
```

🔹 20. center()

👉 Aligns string in center

```
text = "Hi"
print(text.center(10, "*"))   # ****Hi****
```

✅ Final Quick Summary

- Case conversion → lower(), upper(), title()
- Searching → find(), index()
- Checking → isalpha(), isdigit(), isalnum()
- Modifying → replace(), strip()
- Splitting & joining → split(), join()

3Q) Explain about objects, its attributes and its classification as mutable and immutable.

Ans) 🔷 Objects in Python

✅ What is an Object?

An object is a basic unit in Python that contains:

- Data (value)
- Methods (functions)

👉 Everything in Python is an object (numbers, strings, lists, etc.)

🔷 3 Attributes of an Object

Every object has three main attributes:

1️⃣ Identity

👉 Unique address of the object in memory

```
x = 10
print(id(x))
```

2️⃣ Type

👉 Tells what kind of object it is

```
x = 10
print(type(x))   # <class 'int'>
```

3️⃣ Value

👉 The actual data stored in the object

```
x = 10
print(x)   # 10
```

🔷 Objects of Built-in Data Types

🔹 Immutable Objects

👉 Cannot be changed after creation

Examples:

- int
- float
- str
- tuple

Example

```
x = 10
x = x + 5   # new object created
```

🔹 Mutable Objects

👉 Can be changed after creation

Examples:

- list
- set
- dict

Example

```
lst = [1, 2, 3]
lst.append(4)   # same object modified
```

✅ Quick Summary

- Object = data + methods
- 3 attributes:
  - Identity (id)
  - Type
  - Value
- Built-in objects:
  - Immutable → cannot change
  - Mutable → can change

4Q) Explain about Regular Expressions and its functions with example.

Ans)

⇒ Regular Expression in python:-

→ Regular expression is a sequence of characters that forms a search pattern.

→ RegEx can be used to check if a string contains a specified search pattern.

→ Python has a built-in function called 're', which can be used to work with Regular expression.

ex:-

① search the string to see if the string starts with "The" and ends with "Spain".

```
import re
txt = "The rain in Spain"
x = re.search("^The.*Spain$", txt)
if x:
    print("Yes! we have a match")
else:
    print("No match")
```

O/P:-

Yes! we have a match

⇒ RegEx functions:-

① findall():-

Returns a list containing all matches of a pattern in a given string.

Syntax:- re.findall(pattern, string)

→ ex:-

```
txt = "cat bat rat"
result = re.findall(r"\w+at", txt)
print(result)
```

O/P:-

['cat', 'bat', 'rat']

② search():-

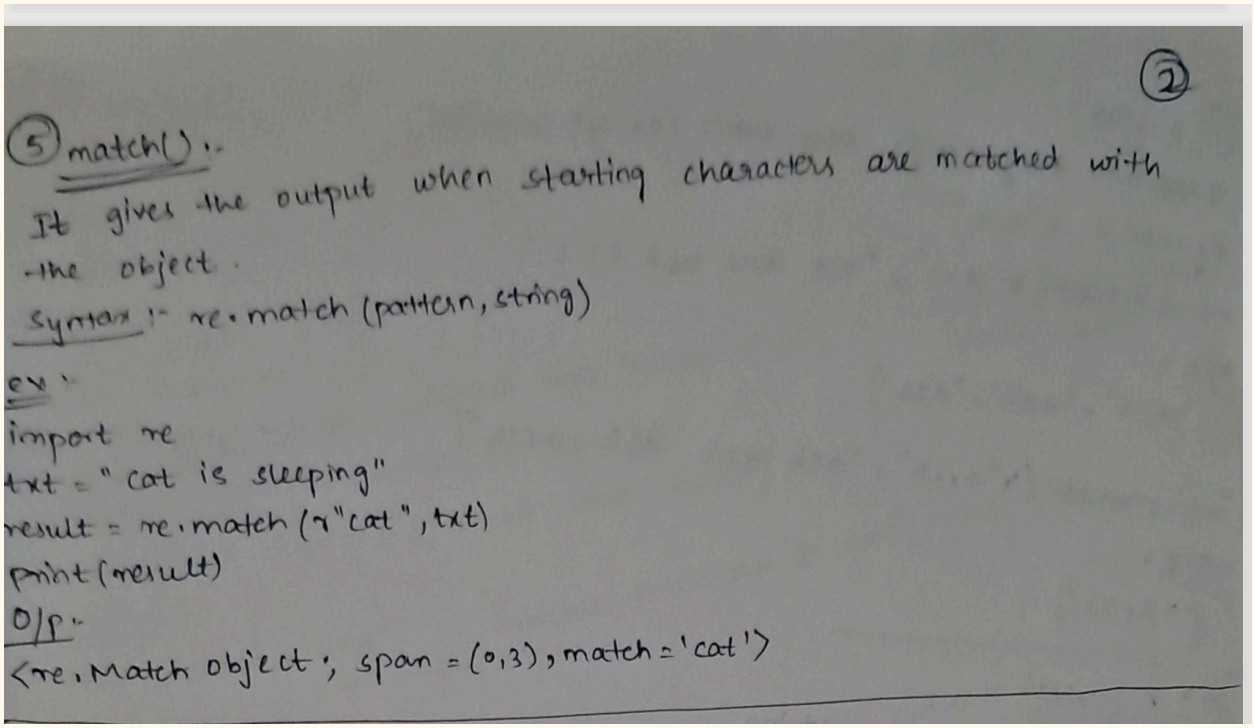
It is used to find the first occurance of a pattern anywhere in the string.

Syntax:- re.search(pattern, string)

→ import re
txt = "I have 2 apples and 3 banaras"
result = re.search(r"\d+", txt)
print(result)
O/P :-
<re.Match object: span(7,8), match='2'>
→ It finds the first number only (2), not all numbers.

(3) split() :-
It is used Returns a list where the string has been split at each match.
Syntax :- re.split(pattern, string, maxsplit=0)
ex :- split by space.
import re
txt = "apple banana mango"
result = re.split(r"\s+", txt)
print(result)
O/P :-
['apple', 'banana', 'mango']
→ \s+ means one or more spaces.

(4) sub() :-
Replace one or many matches with a string.
Syntax :- re.sub(pattern, replacement, string, count=0)
→ import re
txt = "I have 2 apples and 3 banans"
result = re.sub(r"\d+", "#", txt)
print(result)
O/P :-
I have # apples and # bananas.
→ All numbers are replaced with #



(2)

(5) match() :-

It gives the output when starting characters are matched with the object.

Syntax :- re.match(pattern, string)

ex :-

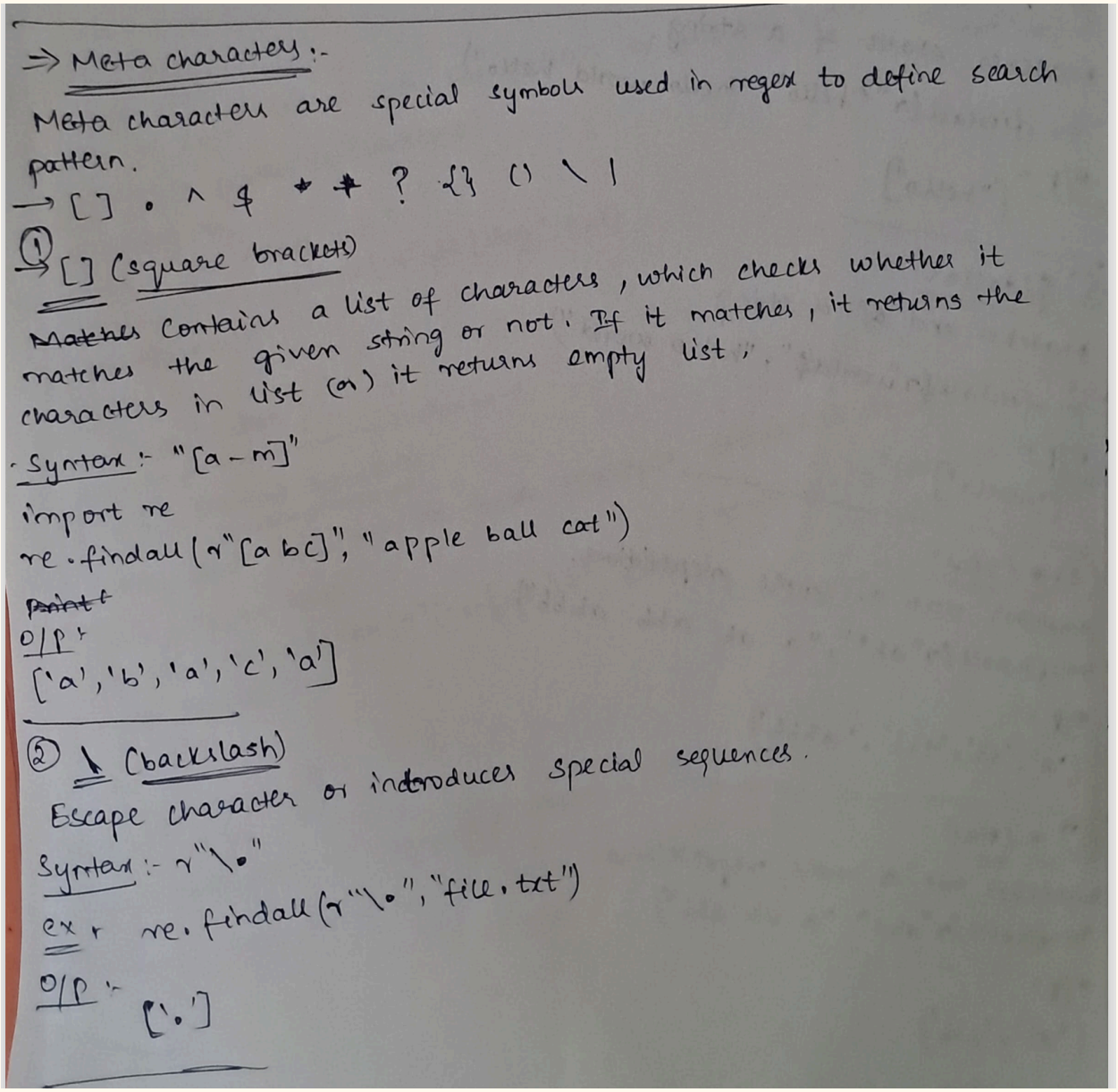
```
import re
txt = "cat is sleeping"
result = re.match(r"cat", txt)
print(result)
```

O/P :-

<re.Match object; span = (0,3), match = 'cat'>

5Q) Explain about meta characters and its functions with examples.

Ans)



⇒ Meta characters :-

Meta characters are special symbols used in regex to define search pattern.

→ [ ] . ^ $ * + ? {} () \ |

① [] (square brackets)

Contains a list of characters, which checks whether it matches the given string or not. If it matches, it returns the characters in list (or) it returns empty list.

Syntax :- "[a-m]"

```
import re
re.findall(r"[abc]", "apple ball cat")
```

O/P :-

```
['a','b','a','c','a']
```

② \ (backslash)

Escape character or introduces special sequences.

Syntax :- r"\."

ex :- re.findall(r"\.", "file.txt")

O/P :- ['.']

③ . (dot)
Matches any single character (except newline).
Syntax: r"a.b"
re.findall(r"a.b", "acb aab a1b accb")
O/P:
['acb', 'aab', 'a1b']

re.findall(r"a..b", "acb aab a1b accb")
O/P:
['accb']

---

④ ^ (caret)
Matches start of a string.
re.findall(r"^hello", "hello world hello")
O/P: ["hello"]

---

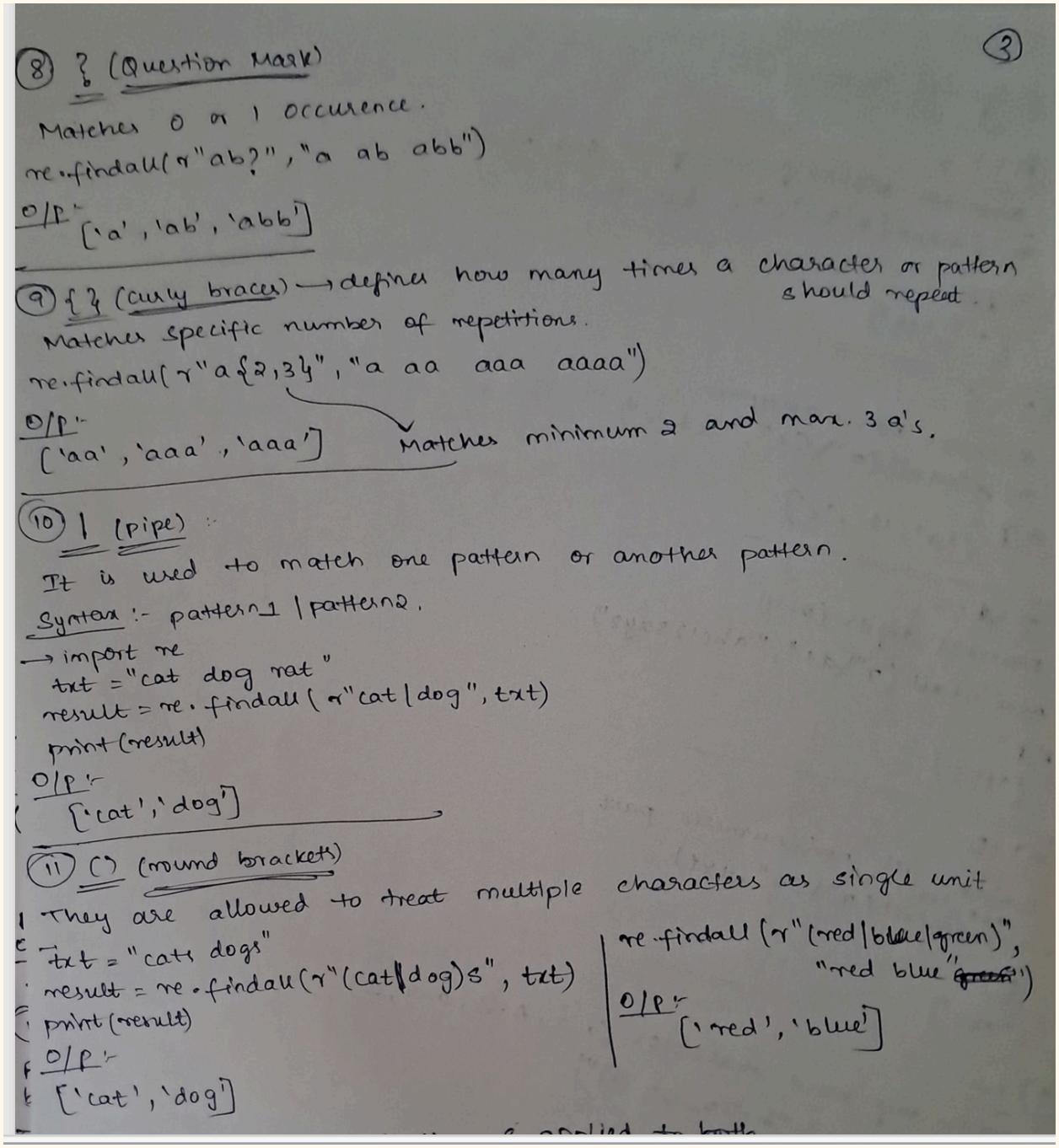
⑤ $ (dollar)
Matches end of a string
re.findall(r"world$", "hello world")
O/P:
['world']

---

⑥ * (star)
Matches zero or more repetitions.
re.findall(r"ab*", "a ab abb abbb")
O/P:
['a', 'ab', 'abb', 'abbb']

---

⑦ + (plus)
Matches one or more repetitions
re.findall(r"ab+", "a ab abb")



(8) ? (Question Mark) (3)

Matches 0 or 1 occurence.

```
re.findall(r"ab?","a ab abb")
```

O/P:-
['a','ab','abb']

(9) {} (curly braces) → define how many times a character or pattern should repeat.

Matches specific number of repetitions.

```
re.findall(r"a{2,3}","a aa aaa aaaa")
```

O/P:-
['aa','aaa','aaa'] Matches minimum 2 and max. 3 a's.

(10) | (pipe)

It is used to match one pattern or another pattern.

Syntax :- pattern1 | pattern2.

```
→ import re
  txt = "cat dog rat"
  result = re.findall(r"cat|dog", txt)
  print(result)
```

O/P:-
['cat','dog']

(11) () (round brackets)

They are allowed to treat multiple characters as single unit

```
txt = "cats dogs"
result = re.findall(r"(cat|dog)s", txt)
print(result)
```

O/P:-
['cats','dogs']

```
re.findall(r"(red|blue|green)",
           "red blue green")
```

O/P:-
['red','blue']

6Q) Explain about special sequences in regular expressions.

Ans)

→ Special sequence :-

(1) \s :-
Matches whitespaces (space, tab, newline)
re.findall(r"\s", "a b\tc")

O/P :-
[' ', '\t']

(2) \S :-
Matches non-whitespace
re.findall(r"\S+", "a b c")

O/P :- ['a', 'b', 'c']

(3) \d :-
Matches digits (0-9)
re.findall(r"\d+", "abc123xyz")

O/P :-
['123']

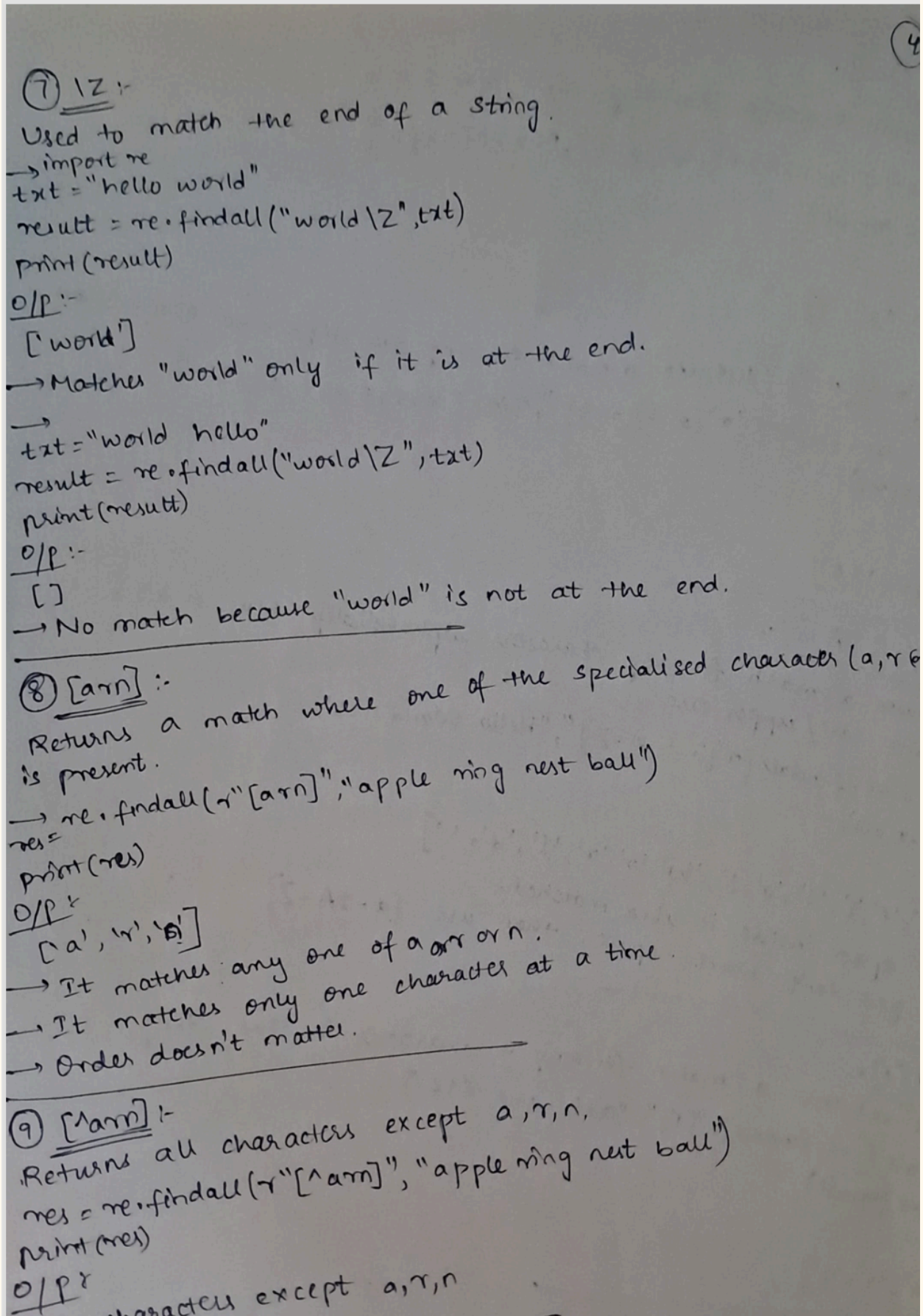
(4) \D :-
Returns all non-numeric part.
txt = "abc123"
→ re.findall(r"\D+", txt)

O/P :-
['abc']

(5) \w :-
Matches any word character (a-z, A-Z, 0-9, _)
re.findall(r"\w+", "hello_123 @")

O/P :-
["hello_123"]

(6) \W :- Matches any non-word characters (@, #, $, %



(7) \Z :-
Used to match the end of a string.

```
import re
txt = "hello world"
result = re.findall("world\Z",txt)
print(result)
```

O/P :-
['world']
→ Matches "world" only if it is at the end.

```
txt = "world hello"
result = re.findall("world\Z",txt)
print(result)
```

O/P :-
[]
→ No match because "world" is not at the end.

(8) [arn] :-
Returns a match where one of the specified characters (a,r,n) is present.

```
res = re.findall(r"[arn]","apple ring nest ball")
print(res)
```

O/P :-
['a','r','n','n']
→ It matches any one of a or r or n.
→ It matches only one character at a time.
→ Order doesn't matter.

(9) [^arn] :-
Returns all characters except a,r,n.

```
res = re.findall(r"[^arn]","apple ring nest ball")
print(res)
```

O/P :-
characters except a,r,n

(11) [0-9] :-
Returns a match for any digit b/w 0 & 9.

```
result = re.findall(r"[0-9]", "abc123xyz")
print(result)
```

O/P :-

```
['1', '2', '3']
```

(12) [0-5][0-9] :- Returns a match for any two digit num. b/w 00 & 59.

```
res = re.findall(r"[0-5][0-9]", "Time: 23:45 and 67:89")
print(res)
```

O/P :-

```
['23', '45']
```

(13) [a-z A-Z] :-
Returns a match for any characters alphabetically b/w a & z, lower case / upper case.

```
res = re.findall(r"[a-z A-Z]", "Hello World 123")
print(res)
```

O/P :-

```
['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', ' ']
```

The space inside is also matched.
If you don't want space, then use [a-zA-Z]

(14) [+] :-
It returns a match for any + characters in the string.

```
result = re.findall(r"[+]", "a+b c+d-e+f")
print(result)
```

O/P :-

```
['+', '+', '+']
```

7Q) Explain about Lists,creation and accessing of lists.

Ans) 🔷 Lists in Python

✅ What is a List?

A list is a collection of elements stored in a single variable.

👉 Lists can store:

- Different data types
- Duplicate values
- Ordered elements

🔹 Example

lst = [1, 2, 3, 4]

print(lst)

🔷 Characteristics of Lists

- Ordered (maintains order)
- Mutable (can be changed)
- Allows duplicates

- Can store different data types

lst = [10, "Python", 3.5, True]

🔷 Creation of Lists

1️⃣ Using Square Brackets [ ]

lst = [1, 2, 3]

2️⃣ Empty List

lst = []

3️⃣ Using list() Function

lst = list((1, 2, 3))

4️⃣ Nested List

👉 List inside another list

lst = [1, 2, [3, 4]]

5️⃣ Using range()

```
lst = list(range(1, 6))
print(lst)   # [1, 2, 3, 4, 5]
```

🔷 Accessing Elements in List

🔹 1. Indexing

👉 Access elements using position

```
lst = [10, 20, 30, 40]

print(lst[0])   # 10
print(lst[2])   # 30
```

🔹 2. Negative Indexing

👉 Access from end

```
print(lst[-1])   # 40
print(lst[-2])   # 30
```

🔹 3. Slicing

👉 Get a portion of list

Syntax:

list[start : end : step]

Example

lst = [10, 20, 30, 40, 50]

```
print(lst[1:4])   # [20, 30, 40]
print(lst[:3])    # [10, 20, 30]
print(lst[::2])   # [10, 30, 50]
```

🔹 4. Accessing Nested List

lst = [1, 2, [3, 4]]

```
print(lst[2])     # [3, 4]
print(lst[2][1])  # 4
```

🔷 Example Program

```
lst = [5, 10, 15, 20]


print("First element:", lst[0])

print("Last element:", lst[-1])

print("Middle elements:", lst[1:3])
```

✅ Final Summary

- List = collection of elements
- Created using [ ] or list()
- Supports different data types
- Access using:
  - Indexing
  - Negative indexing
  - Slicing
- Lists are mutable (can change)

8Q) Explain List operations.

Ans) 🔷 List Operations in Python

List operations are actions performed on lists such as joining, repeating, checking elements, etc.

🔹 1. Concatenation (+)

👉 Combines two lists

a = [1, 2]

b = [3, 4]

```
print(a + b)   # [1, 2, 3, 4]
```

🔹 2. Repetition (*)

👉 Repeats list elements

lst = [1, 2]

```
print(lst * 3)   # [1, 2, 1, 2, 1, 2]
```

🔹 3. Membership (in, not in)

👉 Checks if element exists

lst = [10, 20, 30]

```
print(20 in lst)      # True
print(40 not in lst)  # True
```

🔹 4. Indexing

👉 Access single element

lst = [10, 20, 30]

```
print(lst[0])   # 10
print(lst[-1])  # 30
```

🔹 5. Slicing

👉 Access multiple elements

lst = [10, 20, 30, 40]

```
print(lst[1:3])   # [20, 30]
print(lst[::-1])  # [40, 30, 20, 10]
```

🔹 6. Updating Elements

👉 Modify list values

```
lst = [1, 2, 3]

lst[1] = 10
print(lst)   # [1, 10, 3]
```

🔹 7. Deleting Elements

- 🔸 Using del

```
lst = [1, 2, 3]

del lst[1]
print(lst)   # [1, 3]
```

🔸 Using remove()

👉 Removes specific value

lst = [1, 2, 3, 2]

```
lst.remove(2)
print(lst)   # [1, 3, 2]
```

🔸 Using pop()

👉 Removes by index

lst = [10, 20, 30]

```
lst.pop()
print(lst)   # [10, 20]
```

🔹 8. Length of List

lst = [1, 2, 3, 4]

```
print(len(lst))   # 4
```

🔹 9. Iteration (Looping)

```
lst = [10, 20, 30]

for i in lst:
    print(i)
```

🔹 10. Comparison of Lists

```
print([1, 2] == [1, 2])   # True
print([1, 2] < [1, 3])    # True
```

🔹 11. Finding Max and Min

```
lst = [5, 2, 9]

print(max(lst))   # 9
print(min(lst))   # 2
```

🔹 12. Sum of Elements

```
lst = [1, 2, 3]


print(sum(lst))   # 6
```

🔹 13. Sorting List

```
lst = [3, 1, 2]


lst.sort()
print(lst)   # [1, 2, 3]
```

🔹 14. Reversing List

```
lst = [1, 2, 3]


lst.reverse()
print(lst)   # [3, 2, 1]
```

🔹 15. Copying List

lst1 = [1, 2, 3]

lst2 = lst1.copy()

print(lst2)

✅ Final Summary

- + → Concatenation
- * → Repetition
- in → Membership
- Indexing & slicing → Access elements
- del, remove(), pop() → Delete elements
- sort(), reverse() → Modify order
- len(), max(), min() → Built-in operations

9Q) Explain List methods.

Ans) 🔷 List Methods in Python

👉 List methods are built-in functions used to modify lists

⚠️ Important:

- List methods change the original list (mutable)

🔹 1. append()

👉 Adds element at the end

```
lst = [1, 2, 3]

lst.append(4)


print(lst)   # [1, 2, 3, 4]
```

🔹 2. extend()

👉 Adds multiple elements (another list)

```
lst = [1, 2]

lst.extend([3, 4])


print(lst)   # [1, 2, 3, 4]
```

🔹 3. insert()

👉 Inserts element at specific position

```
lst = [1, 2, 3]

lst.insert(1, 10)


print(lst)   # [1, 10, 2, 3]
```

🔹 4. remove()

👉 Removes first occurrence of value

```
lst = [1, 2, 3, 2]

lst.remove(2)


print(lst)   # [1, 3, 2]
```

🔹 5. pop()

👉 Removes element using index
👉 Returns removed element

```
lst = [10, 20, 30]

x = lst.pop()


print(lst)   # [10, 20]

print(x)     # 30
```

🔹 6. clear()

👉 Removes all elements

```
lst = [1, 2, 3]

lst.clear()


print(lst)   # []
```

🔹 7. index()

👉 Returns index of first occurrence

```
lst = [10, 20, 30]
print(lst.index(20))   # 1
```

🔹 8. count()

👉 Counts occurrences

```
lst = [1, 2, 2, 3]
print(lst.count(2))   # 2
```

🔹 9. sort()

👉 Sorts list (ascending by default)

```
lst = [3, 1, 2]
lst.sort()

print(lst)   # [1, 2, 3]
```

👉 Descending:

```
lst.sort(reverse=True)
```

🔹 10. reverse()

👉 Reverses list

```
lst = [1, 2, 3]

lst.reverse()


print(lst)   # [3, 2, 1]
```

🔹 11. copy()

👉 Creates a copy of list

```
lst1 = [1, 2, 3]

lst2 = lst1.copy()


print(lst2)
```

🔷 Quick Table Summary

Method	Purpose

append()	Add single element
extend()	Add multiple elements
insert()	Add at specific index
remove()	Remove by value
pop()	Remove by index
clear()	Remove all
index()	Find position
count()	Count elements
sort()	Sort list
reverse()	Reverse list

copy() Copy list

✅ Final Points

- Lists are mutable → methods modify original list
- Most methods do not return new list (except pop())

10Q) Explain List Comprehension.

Ans) 🔷 List Comprehensions in Python

✅ Definition

A list comprehension is a short and simple way to create lists using a single line of code.

👉 It replaces loops with a compact syntax.

🔷 Syntax

[expression for item in iterable]

👉 With condition:

[expression for item in iterable if condition]

🔷 Basic Example

- Without list comprehension

```
lst = []
for i in range(5):
    lst.append(i)

print(lst)   # [0, 1, 2, 3, 4]
```

- With list comprehension

```
lst = [i for i in range(5)]
print(lst)
```

🔷 Examples

1️⃣ Squares of numbers

```
lst = [i*i for i in range(5)]
print(lst)   # [0, 1, 4, 9, 16]
```

2️⃣ Even numbers only

```
lst = [i for i in range(10) if i % 2 == 0]
print(lst)   # [0, 2, 4, 6, 8]
```

3️⃣ Convert to uppercase

```
words = ["hello", "python"]

lst = [w.upper() for w in words]
print(lst)   # ['HELLO', 'PYTHON']
```

4️⃣ Using condition (if-else)

```
lst = ["Even" if i % 2 == 0 else "Odd" for i in range(5)]
print(lst)
```

5️⃣ Nested list comprehension

```
lst = [(i, j) for i in range(2) for j in range(2)]
print(lst)
```

🔷 Advantages

- Short and readable code
- Faster than normal loops
- Easy to write

🔷 Important Points

- Used to create lists quickly
- Can include conditions
- Can replace loops + append()

🔷 Example Program

```
numbers = [1, 2, 3, 4, 5]

squares = [n*n for n in numbers if n > 2]

print(squares)   # [9, 16, 25]
```

✅ Final Summary

- List comprehension = shortcut for creating lists
- Syntax:
  - [expression for item in iterable]
  - [expression for item in iterable if condition]
- Can include:
  - Conditions
  - Nested loops
  - If-else

---

11) Explain about Tuples and its characteristics in detail.

Ans) 🔷 Tuples in Python

✅ What is a Tuple?

A tuple is a collection of elements stored in a single variable.

👉 Similar to a list, but:

- Immutable (cannot be changed)
- Uses parentheses ( )

🔹 Example

t = (1, 2, 3)

print(t)

🔷 Characteristics of Tuples

- Ordered (elements have fixed order)
- Immutable (cannot modify elements)
- Allows duplicate values
- Can store different data types

t = (10, "Python", 3.5)

🔷 Creation of Tuples

1️⃣ Using parentheses

t = (1, 2, 3)

2️⃣ Without parentheses (packing)

t = 1, 2, 3

3️⃣ Single element tuple

👉 Must use comma

t = (5,) # correct

4️⃣ Empty tuple

t = ()

5️⃣ Using tuple() function

```
t = tuple([1, 2, 3])
```

🔷 Accessing Elements

- 🔹 Indexing

```
t = (10, 20, 30)
print(t[0])   # 10
```

- 🔹 Negative Indexing

```
print(t[-1])   # 30
```

- 🔹 Slicing

```
t = (10, 20, 30, 40)
print(t[1:3])   # (20, 30)
```

🔷 Tuple Operations

- 🔹 Concatenation

```
t1 = (1, 2)
t2 = (3, 4)


print(t1 + t2)   # (1, 2, 3, 4)
```

- Repetition

```
t = (1, 2)
print(t * 2)   # (1, 2, 1, 2)
```

- Membership

```
t = (1, 2, 3)
print(2 in t)   # True
```

🔷 Tuple Methods

👉 Tuples have only two methods

- count()

```
t = (1, 2, 2, 3)
print(t.count(2))   # 2
```

- index()

```
t = (10, 20, 30)
print(t.index(20))   # 1
```

🔷 Important Concept: Immutability

```
t = (1, 2, 3)
```

t[1] = 10 # ❌ Error

👉 But if tuple contains list:

t = (1, [2, 3])

t[1][0] = 10 # ✔ allowed

🔷 Packing and Unpacking

- 🔹 Packing

t = 1, 2, 3

- 🔹 Unpacking

a, b, c = (1, 2, 3)

print(a, b, c)

🔷 Example Program

t = (5, 10, 15)

print("First:", t[0])

print("Last:", t[-1])

print("Slice:", t[1:])

✅ Final Summary

- Tuple = ordered collection
- Immutable (cannot change)
- Created using ( )
- Supports indexing & slicing
- Only 2 methods: count(), index()

---

12) Explain Functions in python.

Ans)

🔷 Functions in Tuples (Built-in Functions)

👉 These are **not tuple methods**, but **built-in functions** that work on tuples.

🔹 1. len()

✅ Purpose

Returns number of elements in tuple

t = (1, 2, 3, 4)

print(len(t)) # 4

🔹 2. max()

✅ **Purpose**

Returns largest element

t = (5, 2, 9)

print(max(t)) # 9

🔹 3. min()

✅ **Purpose**

Returns smallest element

t = (5, 2, 9)

print(min(t)) # 2

🔹 4. sum()

✅ **Purpose**

Returns sum of elements

t = (1, 2, 3)

print(sum(t)) # 6

🔹 5. sorted()

✅ **Purpose**

Returns sorted list (not tuple)

t = (3, 1, 2)

print(sorted(t)) # [1, 2, 3]

👉 Output is **list**, not tuple

🔹 6. tuple()

✅ Purpose

Converts iterable into tuple

lst = [1, 2, 3]

t = tuple(lst)

print(t) # (1, 2, 3)

🔹 7. any()

✅ Purpose

Returns True if any element is True

t = (0, False, 5)

print(any(t)) # True

🔹 8. all()

✅ Purpose

Returns True if all elements are True

```
t = (1, 2, 3)

print(all(t))   # True
```

🔹 9. enumerate()

✅ Purpose

Returns index + value pairs

```
t = ('a', 'b', 'c')


for i, v in enumerate(t):
    print(i, v)
```

🔹 10. reversed()

✅ Purpose

Returns reversed iterator

```
t = (1, 2, 3)
```

```
print(list(reversed(t)))   # [3, 2, 1]
```

🔷 Important Points

- These are **general functions**, not specific to tuples
- Work on many data types (list, string, tuple, etc.)
- Do not modify tuple (since tuples are immutable)

🔷 Final Summary

👉 Functions used with tuples:

- len() → length
- max() → largest
- min() → smallest
- sum() → total
- sorted() → sorted list
- tuple() → create tuple
- any(), all() → logical check
- enumerate() → index + value
- reversed() → reverse order

---

13) Give key points for tuple.

Ans)

🔷 Key Points of Tuples

🔹 1. Immutability

👉 Tuples **cannot be changed** after creation

```
t = (1, 2, 3)
t[0] = 10   # ❌ Error
```

✔ But if tuple contains mutable object:

```
t = (1, [2, 3])
t[1][0] = 10   # ✔ Allowed
```

🔹 2. Packing vs Unpacking

✅ **Packing**

👉 Storing multiple values into a tuple

```
t = 1, 2, 3
```

✅ **Unpacking**

👉 Extracting values

```
a, b, c = (1, 2, 3)
print(a, b, c)
```

🔹 3. Swapping (Without Temp Variable)

👉 Tuples make swapping easy

```
a = 5
```

```
b = 10

a, b = b, a

print(a, b)   # 10 5
```

🔹 4. Single Element Trap ⚠️

👉 Writing (5) is NOT a tuple

```
t = (5)

print(type(t))   # int ❌
```

✔ Correct:

```
t = (5,)

print(type(t))   # tuple ✔
```

🔹 5. Tuple is Hashable

👉 Tuples can be used as **dictionary keys** (if elements are immutable)

```
d = {(1, 2): "value"}

print(d[(1, 2)])
```

⚠️ Not allowed if tuple contains list:

```
t = (1, [2, 3])   # ❌ not hashable
```

🔹 6. Tuple ID Change

👉 Since tuples are immutable, any modification creates **new object**

```
t = (1, 2)
print(id(t))


t = t + (3,)
print(id(t))   # Different ID
```

🔹 7. Tuple Multiplication Trap ⚠️

👉 Works fine for normal elements:

```
t = (1, 2)
print(t * 3)   # (1, 2, 1, 2, 1, 2)
```

⚠️ Problem with mutable elements:

```
t = ([0],) * 3
t[0][0] = 5


print(t)   # ([5], [5], [5])
```

👉 All elements refer to **same list**

🔹 8. Handling Nested Tuples

👉 Access elements using multiple indices

```
t = (1, (2, 3), (4, 5))


print(t[1])      # (2, 3)
print(t[1][0])   # 2
```

🔹 **Loop through nested tuple**

```
for item in t:
    print(item)
```

✅ Final Quick Summary

- Immutable → cannot change
- Packing / Unpacking supported
- Easy swapping
- Single element needs comma (x,)
- Hashable → usable as dict keys
- New object created on modification
- Be careful with multiplication (mutable elements)
- Nested tuples accessed using multiple indices

---

14) Distinguish between Tuple and List.

Ans)

Feature	List	Tuple
Definition	Collection of elements	Collection of elements
Syntax	[ ]	( )
Mutability	✅ Mutable (can change)	❌ Immutable (cannot change)
Example	[1, 2, 3]	(1, 2, 3)
Modification	Allowed	Not allowed
Methods	Many methods	Only 2 methods
Performance	Slower	Faster
Memory usage	More	Less
Hashable	❌ Not hashable	✅ Hashable (if elements immutable)
Used as dictionary key	❌ No	✅ Yes
Safety	Less safe (can change)	More safe (fixed data)

🔷 Examples

🔹 List (Mutable)

```
lst = [1, 2, 3]

lst[0] = 10

print(lst)   # [10, 2, 3]
```

🔹 Tuple (Immutable)

```
t = (1, 2, 3)

t[0] = 10   # ❌ Error
```

🔷 Key Differences Explained

🔹 1. Mutability

- Lists can be changed
- Tuples cannot be changed

🔹 2. Methods

- List → append(), remove(), sort(), etc.
- Tuple → only count(), index()

🔹 3. Performance

- Tuples are **faster** (fixed size)

- Lists are **slower** (dynamic)

🔹 4. Use Case

- List → when data changes frequently
- Tuple → when data should remain constant

🔷 When to Use What?

👉 Use **List**:

- When you need to modify data
- Example: storing marks, items, etc.

👉 Use **Tuple**:

- When data should not change
- Example: coordinates, fixed values

✅ Final Summary

- List = mutable, flexible
- Tuple = immutable, fixed
- Lists → more features
- Tuples → more efficient

---

15) Explain about Sets in python.

Ans)

🔷 Sets in Python

✅ What is a Set?

A **set** is a collection of **unique elements.**

👉 Important:

- No duplicate values
- Unordered (no fixed position)

🔹 Example

s = {1, 2, 3, 4}

print(s)

🔷 Characteristics of Sets

- Unordered (no indexing)
- No duplicate elements
- Mutable (can add/remove elements)
- Cannot contain mutable items (like lists)

s = {1, 2, 2, 3}

print(s) # {1, 2, 3}

🔷 Creation of Sets

🔹 1. Using { }

```
s = {1, 2, 3}
```

🔹 2. Empty Set ⚠️

```
s = set()   # correct
```

❌ {} creates dictionary, not set

🔹 3. Using set() function

```
s = set([1, 2, 3])
```

🔷 Accessing Elements

👉 Sets are **unordered**, so:

- ❌ No indexing
- ❌ No slicing

✔ Access using loop:

```
s = {1, 2, 3}


for i in s:
    print(i)
```

🔷 Set Operations

🔹 1. Union (|)

👉 Combines elements

a = {1, 2}

b = {3, 4}

```
print(a | b)   # {1, 2, 3, 4}
```

🔹 **2. Intersection (&)**

👉 Common elements

```
print(a & b)
```

🔹 **3. Difference (-)**

a = {1, 2, 3}

b = {2, 3}

```
print(a - b)   # {1}
```

🔹 **4. Symmetric Difference (^)**

👉 Elements not common

```
print(a ^ b)
```

🔷 Set Methods

🔹 add()

s = {1, 2}

s.add(3)

🔹 remove()

s.remove(2)

🔹 discard()

👉 No error if element not present

s.discard(5)

🔹 pop()

👉 Removes random element

s.pop()

🔹 clear()

s.clear()

🔹 update()

👉 Add multiple elements

s.update([4, 5])

🔷 Important Points

- No duplicates allowed
- No indexing or slicing
- Elements must be immutable (int, string, tuple)
- Used for mathematical operations

🔷 Example Program

a = {1, 2, 3}

b = {2, 3, 4}

print("Union:", a | b)

print("Intersection:", a & b)

print("Difference:", a - b)

✅ Final Summary

- Set = unordered collection of unique elements
- Created using {} or set()
- No duplicates, no indexing
- Supports union, intersection, difference
- Mutable but elements must be immutable

---

16) Distinguish between sets vs lists vs tuples.

Ans)

Feature	List	Tuple	Set
Syntax	[ ]	( )	{ }
Example	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}
Order	✅ Ordered	✅ Ordered	❌ Unordered
Mutability	✅ Mutable	❌ Immutable	✅ Mutable
Duplicates	✅ Allowed	✅ Allowed	❌ Not allowed
Indexing	✅ Yes	✅ Yes	❌ No
Slicing	✅ Yes	✅ Yes	❌ No
Methods	Many	Only 2	Many
Performance	Medium	Fast	Fast (for membership)

Hashable	❌ No	✅ Yes (if elements immutable)	❌ No
Use case	Dynamic data	Fixed data	Unique elements

🔷 Key Differences Explained

🔹 1. Order

- List & Tuple → maintain order
- Set → no order (random)

🔹 2. Mutability

- List & Set → can change
- Tuple → cannot change

🔹 3. Duplicates

- List & Tuple → allow duplicates
- Set → removes duplicates automatically

```
s = {1, 2, 2, 3}

print(s)   # {1, 2, 3}
```

🔹 4. Indexing

- List & Tuple → support indexing

- Set → does NOT support indexing

🔹 5. Use Cases

👉 **List**

- When data changes frequently
- Example: student marks

👉 **Tuple**

- When data should not change
- Example: coordinates

👉 **Set**

- When unique elements are needed
- Example: removing duplicates

🔷 Simple Examples

🔹 List

```
lst = [1, 2, 3]

lst.append(4)
```

🔹 Tuple

```
t = (1, 2, 3)

# cannot modify
```

🔹 **Set**

s = {1, 2, 3}

s.add(4)

✅ Final Summary

- **List** → ordered, mutable, duplicates allowed
- **Tuple** → ordered, immutable
- **Set** → unordered, unique elements only

---

17) Explain Set Operation methods.

Ans)

🔷 Set Operation Methods (Simple Explanation)

- Sets in Python are often used to perform **mathematical operations**
- These operations are similar to what we study in **math (Venn diagrams)**

🔷 Common Set Operations

- **Union** → combine all elements
- **Intersection** → common elements

- **Difference** → elements in one set but not the other
- **Symmetric Difference** → elements in either set but not both

🔷 Example Sets

dataScientist = set(['Python', 'R', 'SQL', 'Git', 'Tableau', 'SAS'])

dataEngineer = set(['Python', 'Java', 'Scala', 'Git', 'SQL', 'Hadoop'])

👉 These are two sets with different skills

🔷 Union (Very Important)

- **Union means combining both sets**
- It includes:
  - Elements in first set
  - Elements in second set
  - Elements common in both
- Final result has **only unique values (no duplicates)**

🔷 How to Perform Union

Method 1: Using union()

dataScientist.union(dataEngineer)

Method 2: Using |

dataScientist | dataEngineer

👉 Both give same result

🔷 Output Meaning

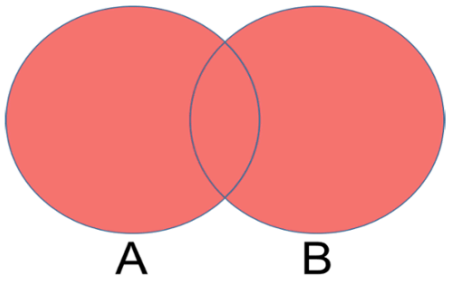
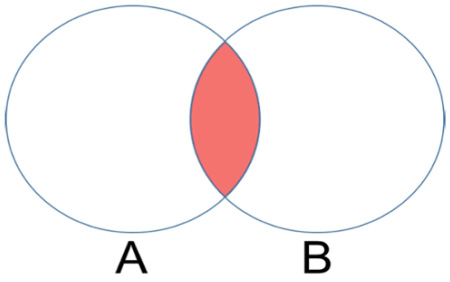
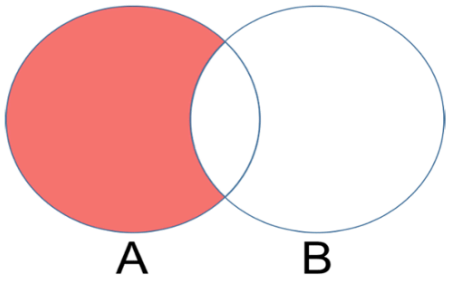
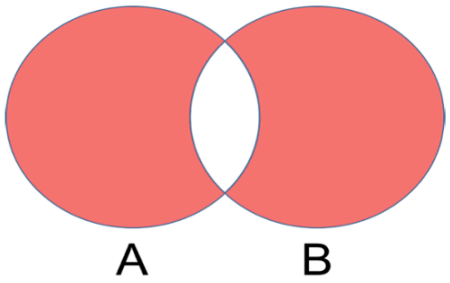
- The result contains **all unique skills from both sets**
- Duplicate values like "Python", "SQL" appear only once

🔷 Simple Understanding (Layman)

- Think of two groups of people
- Union = **everyone from both groups together (no repeats)**

🔷 Final Point

- In Venn diagram:
  - Union = **full combined area (both circles together)**

Set Operation	Venn Diagram	Interpretation
Union		$A \cup B$, is the set of all values that are a member of A, or B, or both.
Intersection		$A \cap B$, is the set of all values that are members of both A and B.
Difference		$A \setminus B$, is the set of all values of A that are not members of B
Symmetric Difference		$A \triangle B$, is the set of all values which are in one of the sets, but not both.

18) Explain about Frozen sets.

Ans) 🔷 Frozenset in Python

✅ What is a Frozenset?

👉 A frozenset is just like a set, but:

- ❌ Immutable (cannot be changed)
- ✅ Contains unique elements only

🔹 Example

```
fs = frozenset([1, 2, 3, 2])

print(fs)   # frozenset({1, 2, 3})
```

👉 Duplicate values are automatically removed

🔷 Key Characteristics

- Unordered (no indexing)
- No duplicates
- Immutable (cannot add/remove elements)
- Can be used as dictionary keys (hashable)

🔷 Creating a Frozenset

```
fs = frozenset([1, 2, 3])
```

👉 You must use frozenset() function
👉 {} cannot be used

🔷 Difference: Set vs Frozenset

Feature	Set	Frozenset
Add/Remove	Allowed	❌ Not allowed
Hashable	❌ No	✅ Yes
Syntax	{}	frozenset()

🔷 Not Allowed Operations

```
fs = frozenset([1, 2, 3])

fs.add(4)      # ❌ Error
fs.remove(2)   # ❌ Error
```

🔷 Allowed Operations

👉 You can still perform set operations

a = frozenset([1, 2, 3])

b = frozenset([3, 4, 5])

print(a | b) # union

print(a & b) # intersection

🔷 Why Use Frozenset?

👉 When you want:

- Data that should not change
- To use set as a dictionary key
- Better safety (no accidental modification)

🔷 Example Use

d = {frozenset([1, 2]): "value"}

print(d)

🔷 Final Summary

- Frozenset = immutable set
- No duplicates, no order
- Cannot modify after creation
- Supports set operations

- Can be used as dictionary key

---

19) Explain Set comprehension.

Ans)

🔷 Set Comprehension in Python

✅ What is Set Comprehension?

👉 A **set comprehension** is a **short and simple way to create a set** in one line.

👉 Similar to **list comprehension**, but uses { }

🔷 Syntax

{expression for item in iterable}

👉 With condition:

{expression for item in iterable if condition}

🔷 Basic Example

🔹 **Without set comprehension**

s = set()

for i in range(5):

```
    s.add(i)


print(s)   # {0, 1, 2, 3, 4}
```

🔹 **With set comprehension**

```
s = {i for i in range(5)}

print(s)
```

🔷 Examples

1️⃣ Squares of numbers

```
s = {i*i for i in range(5)}

print(s)   # {0, 1, 4, 9, 16}
```

2️⃣ Even numbers only

```
s = {i for i in range(10) if i % 2 == 0}

print(s)   # {0, 2, 4, 6, 8}
```

3️⃣ Remove duplicates

```
lst = [1, 2, 2, 3, 3]

s = {i for i in lst}

print(s)   # {1, 2, 3}
```

4️⃣ Convert to uppercase

```
words = ["hello", "python"]


s = {w.upper() for w in words}

print(s)
```

🔷 Important Points

- Uses { }
- Automatically removes duplicates
- Unordered output
- Faster and shorter than loops

🔷 Difference from List Comprehension

Feature	List Comprehension	Set Comprehension
Syntax	[ ]	{ }
Duplicates	Allowed	❌ Removed

Order	Maintained	❌ Not maintained

🔷 Example Program

numbers = [1, 2, 3, 4, 5]

result = {n*n for n in numbers if n > 2}

print(result) # {9, 16, 25}

✅ Final Summary

- Set comprehension = **shortcut to create sets**
- Syntax: {expression for item in iterable}
- Removes duplicates automatically
- Supports conditions

---

20) Explain Dictionaries in python.

Ans)

🔷 Python Dictionaries (Simple Explanation)

🔹 What is a Dictionary?

- A dictionary stores data in key : value pairs

- Example:

```
thisdict = {
 "brand": "Ford",
 "model": "Mustang",
 "year": 1964
}
```

- Here:
  - "brand" → key
  - "Ford" → value

🔹 Key Features of Dictionary

- Stores data as key : value
- Keys must be unique
- Values can be repeated
- Changeable (you can modify it)
- Ordered (in Python 3.7+ order is maintained)

🔹 Ordered or Unordered?

- Python 3.7+ → Ordered
  - Items stay in the same order you add them
- Older versions → Unordered
  - No fixed order

👉 You cannot use index (like 0,1,2) to access items

🔹 No Duplicate Keys

- You cannot have same key twice

```
thisdict = {
 "year": 1964,
 "year": 2020
}
```

👉 Output:

- Only latest value is kept → "year": 2020

🔹 Accessing Values

Using key

```
print(thisdict["brand"])
```

Using get() (safer)

```
print(thisdict.get("model"))
```

👉 No error if key is missing

🔹 Getting Keys

```
x = thisdict.keys()
```

- Returns all keys
- Automatically updates if dictionary changes

- Changing Values

thisdict["year"] = 2018

👉 Updates value of key

- Using update()

thisdict.update({"year": 2020})

👉 Another way to update values

- Adding New Items

thisdict["color"] = "red"

👉 Adds new key-value pair

🔷 Removing Items

- 1. pop()

thisdict.pop("model")

👉 Removes specific item

- 2. popitem()

thisdict.popitem()

👉 Removes last added item

- 3. del

```
del thisdict["model"]
```

👉 Deletes specific item

🔹 4. Delete Entire Dictionary

```
del thisdict
```

👉 Completely removes dictionary

🔹 5. clear()

```
thisdict.clear()
```

👉 Empties dictionary (but keeps it)

🔷 Simple Understanding

- Dictionary = like a real-life dictionary 📖
- You search using a word (key)
- You get its meaning (value)

✅ Final Summary

- Dictionary = key-value pairs
- Keys → unique
- Values → can repeat
- Mutable (can change)
- No indexing, use keys to access
- Supports add, update, delete operations

---

21) Give Key points in Dictionary.

Ans)

🔷 Key Points of Dictionary

🔹 1. Key–Value Pair Structure

👉 Dictionary stores data as:

key : value

d = {"name": "Akhil", "age": 20}

🔹 2. Keys Must Be Unique

👉 Duplicate keys are not allowed

d = {"a": 1, "a": 2}

Output: {'a': 2}

🔹 3. Values Can Be Duplicate

d = {"a": 1, "b": 1}

🔹 4. Mutable (Changeable)

👉 You can add, update, delete items

d["age"] = 21

🔹 5. Ordered (Python 3.7+)

👉 Maintains insertion order

🔹 6. No Indexing

👉 Cannot access using position

```
d[0]   # ❌ Error
```

🔹 7. Access Using Keys

```
print(d["name"])
```

🔹 8. Keys Must Be Immutable

👉 Allowed:

- int, string, tuple

❌ Not allowed:

- list, set, dictionary

🔹 9. Fast Access

👉 Searching using keys is very fast

🔹 10. Nested Dictionaries Allowed

```
d = {"student": {"name": "Akhil"}}
```

🔹 11. Dynamic Size

👉 Can grow or shrink anytime

🔹 12. Common Methods

- keys() → get keys
- values() → get values
- items() → key-value pairs
- update() → update dictionary
- pop() → remove item

🔷 Final One-Line Summary

👉 **Dictionary = mutable, ordered collection of unique keys mapped to values**

---

22) Write a python program to reverse dictionary, character frequency, merge dictionaries, sort dictionaries by values, max. value key, group words by length, nested dictionary, remove duplicate values, word count, dictionary from 2 lists.

Ans)

🔷 1. Reverse Dictionary

👉 Swap keys and values

d = {"a": 1, "b": 2}

rev = {v: k for k, v in d.items()}

```
print(rev)   # {1: 'a', 2: 'b'}
```

🔷 2. Character Frequency

👉 Count each character in a string

```
text = "hello"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)   # {'h':1, 'e':1, 'l':2, 'o':1}
```

🔷 3. Merge Dictionaries

👉 Combine two dictionaries

```
d1 = {"a": 1}
d2 = {"b": 2}
merged = {**d1, **d2}
print(merged)
```

🔷 4. Sort Dictionary by Values

👉 Sort based on values

```
d = {"a": 3, "b": 1, "c": 2}
```

```
sorted_d = dict(sorted(d.items(), key=lambda x: x[1]))
print(sorted_d)
```

🔷 5. Max Value Key

👉 Find key with highest value

```
d = {"a": 10, "b": 20, "c": 15}

key = max(d, key=d.get)
print(key)   # b
```

🔷 6. Group Words by Length

👉 Group words based on length

```
words = ["hi", "hello", "hey"]

result = {}
for w in words:
    result.setdefault(len(w), []).append(w)
```

```
print(result)

# {2: ['hi'], 5: ['hello'], 3: ['hey']}
```

🔷 7. Nested Dictionary

👉 Dictionary inside another dictionary

```
d = {
  "student1": {"name": "Akhil", "marks": 90},
  "student2": {"name": "Ram", "marks": 85}
}

print(d["student1"]["name"])
```

🔷 8. Remove Duplicate Values

👉 Keep only unique values

```
d = {"a": 1, "b": 2, "c": 1}

result = {}

for k, v in d.items():
  if v not in result.values():
      result[k] = v

print(result)
```

🔷 9. Word Count

👉 Count words in a sentence

```
text = "hi hello hi"
words = text.split()
count = {}
for w in words:
    count[w] = count.get(w, 0) + 1
print(count)
```

🔷 10. Dictionary from 2 Lists

👉 Create dictionary using two lists

```
keys = ["a", "b", "c"]
values = [1, 2, 3]
d = dict(zip(keys, values))
print(d)
```

✅ Final Summary

- Reverse → swap key & value
- Frequency → count occurrences
- Merge → combine dictionaries

- Sort → based on values
- Max key → highest value
- Grouping → based on condition
- Nested → dictionary inside dictionary
- Remove duplicates → filter values
- Word count → count words
- From lists → use zip()

---

23) Explain sets vs Dictionary.

Ans)

🔷 Set vs Dictionary

Feature	Set	Dictionary
Definition	Collection of unique elements	Collection of key–value pairs
Syntax	{1, 2, 3}	{"a": 1, "b": 2}
Data Stored	Only values	Key + Value
Duplicates	❌ Not allowed	❌ Keys not allowed (values allowed)

Order	Unordered	Ordered (Python 3.7+)
Indexing	❌ Not allowed	❌ Not allowed (use keys)
Mutability	✅ Mutable	✅ Mutable
Access	Cannot directly access elements	Access using keys
Use Case	Remove duplicates, set operations	Store related data (key-value)

🔷 Key Differences Explained

🔹 1. Structure

- Set → only elements
- Dictionary → key + value

```
s = {1, 2, 3}
d = {"a": 1, "b": 2}
```

🔹 2. Uniqueness

- Set → all elements must be unique
- Dictionary → keys must be unique (values can repeat)

🔹 3. Accessing Data

- Set → cannot access using index
- Dictionary → access using keys

```
d["a"]   # ✔ works
```

🔹 4. Use Case

👉 Set

- Remove duplicates
- Perform union, intersection

👉 Dictionary

- Store structured data
- Example: student name → marks

🔷 Simple Understanding

- Set → like a bag of unique items
- Dictionary → like a real dictionary 📖
  (word → meaning)

🔷 Example

```
# Set

s = {1, 2, 2, 3}

print(s)   # {1, 2, 3}
```

```
# Dictionary

d = {"name": "Akhil", "age": 20}

print(d["name"])   # Akhil
```

✅ Final Summary

- Set → unique values only
- Dictionary → key-value pairs
- Set → for mathematical operations
- Dictionary → for mapping data

---

24) What is Hashing in Dictionary.

Ans)

🔷 Hashing in Dictionary

✅ What is Hashing?

👉 **Hashing** is a technique used to **convert a key into a number (hash value)**
👉 This number helps Python **store and find data quickly**

🔷 Why Hashing is Used?

- To make **searching very fast**
- Dictionary operations like:

- Access
- Insert
- Delete
  👉 All become **O(1) (very fast)**

🔷 How Hashing Works (Simple)

1. You give a **key**
2. Python applies a **hash function**
3. It generates a **unique number (hash value)**
4. This number decides **where to store the value**

🔹 Example

d = {"name": "Akhil"}

👉 Steps:

- Key = "name"
- Hash function → converts "name" into a number
- Value "Akhil" is stored at that location

🔷 Important Rule

👉 **Keys must be hashable**

🔷 What is Hashable?

👉 A value is hashable if:

- It has a **fixed value (immutable)**

- Its hash value does not change

✅ Hashable Types

- int
- float
- str
- tuple (only if elements are immutable)

```
d = {(1, 2): "value"}   # ✔ valid
```

❌ Not Hashable

- list
- set
- dictionary

```
d = {[1, 2]: "value"}   # ❌ Error
```

🔷 Collision (Basic Idea)

👉 Sometimes two keys may produce the same hash value

👉 Python handles this internally
(no need to worry in exams usually)

🔷 Advantages of Hashing

- Very fast data access
- Efficient storage
- Used in dictionaries and sets

🔷 Simple Analogy

👉 Think of a **library system:**

- Book name (key) → converted to **rack number (hash)**
- You directly go to that rack → fast search

✅ Final Summary

- Hashing converts key → number
- Used for fast access in dictionaries
- Keys must be **immutable (hashable)**
- Enables O(1) operations

---

25) Explain about collisions in python.

Ans)

🔷 What is a Collision?

👉 A **collision** happens when:

Two different keys produce the same hash value

🔹 Simple Idea

Key1 → hash value = 10

Key2 → hash value = 10 ❗

👉 Both keys want to go to the **same memory location**

🔷 Why Collisions Occur?

- Hash function converts data into a number
- But memory locations are limited
- So sometimes **different keys get same hash**

🔷 Example (Conceptual)

d = {}

d["ab"] = 1

d["ba"] = 2

👉 Sometimes "ab" and "ba" may give same hash
👉 This creates a **collision**

🔷 How Python Handles Collisions

👉 Python automatically handles it internally
👉 You don't need to manage it manually

🔹 **Method Used: Hash Table with Probing (Simplified)**

👉 If a position is already occupied:

- Python looks for **another empty place**

🔹 **Easy Understanding**

Think of parking 🚗:

- Slot 10 is assigned
- But already occupied
  👉 Car moves to next free slot (11, 12...)

🔷 Key Point

👉 Even if collision happens:

- Dictionary still works correctly
- Data is stored safely

🔷 Does Collision Affect Performance?

- Normally → ❌ No noticeable effect
- Too many collisions → ⚠️ Slightly slower

🔷 Final Summary

- Collision = two keys → same hash value
- Happens due to limited space
- Python resolves it automatically
- Uses probing / internal techniques
- Rarely affects performance

CODING PROBLEMS ON SETS:

#Q1). Create a Set and Print Elements

```
s={1,2,3,4,5,5}
print(s)
```

Output:

```
{1, 2, 3, 4, 5}
```

#Q2) Add an Element to a Set

```
s={1,2,3,4,5,5}
s.add(6)
print(s)
```

Output:

```
{1, 2, 3, 4, 5, 6}
```

#Q3) Remove an Element

```
s={1,2,3,4,5,5}
s.remove(5)
print(s)
```

Output:

```
{1, 2, 3, 4}
```

#Q4) Find Length of Set

```
s={1,2,3,4,5,5}
print(len(s))
```

Output:

```
5
```

#Q5) Check Element Exists or Not

s={1,2,3,4,5,5}

print(2 in s)

Output:

True

#Q6) Union of Two Sets

s1={1,2,3,4,5,5}

s2={6,7,7,5,3}

print(s1.union(s2))

print(s1|s2)

Output:

{1, 2, 3, 4, 5, 6, 7}

{1, 2, 3, 4, 5, 6, 7}

#Q7) Intersection of Two Sets

s1={1,2,3,4,5,5}

s2={6,7,7,5,3}

print(s1.intersection(s2))

print(s1&s2)

Output:

{3, 5}
{3, 5}

#Q8) Difference of Sets

s1={1,2,3,4,5,5}

s2={6,7,7,5,3,98}

print(s1-s2)

Output:

{1, 2, 4}

#Q9) Symmetric Difference

s1={1,2,3,4,5,5}

s2={6,7,7,5,3,98}

print(s1^s2)

Output:

{1, 98, 2, 6, 7, 4}

#Q10) Remove Duplicates from List

l=[1,2,3,4,5,5]

l=list(set(s1))

print(l)

Output:

[1, 2, 3, 4, 5]

#Q11) Find Common Elements Without Built-in

```
a={1,2,3,4,5}

b={2,3,5,7,8,9}

print(a&b)

#or

common={i for i in a if i in b}

print(common)
```

Output:

{2, 3, 5}

{2, 3, 5}

```
#Q12) Elements Present in Only One Set

a={1,2,3,4,5}

b={2,3,5,7,8,9}

result={x for x in a if x not in b}|{y for y in b if y not in a}

print(result)
```

Output:

{1, 4, 7, 8, 9}

```
#Q13) Subset and Superset

a={1,2}

b={1,2,3,4}

print(a.issubset(b))

print(a.issuperset(b))
```

Output:

```
True
False
```

```
#Q14) Count Unique Words
sentence="Python is easy Python is hard"
words=sentence.split()
unique_words=set(words)
print(len(unique_words))
```

Output:

```
4
```

```
#Q15) First Repeated Element
l=[1,2,2,3,4,4,4,5]
seen=set()
for i in l:
    if i in seen:
        print("First repeated element",i)
        break
    seen.add(i)
```

Output:

```
First repeated element 2
```

#Q16) Check Common Elements in Lists

```
l1=[1,2,3,4,5,6]

l2=[5,6,8,3,8,1]

result=bool(set(l1)&set(l2))

print(result)
```

Output:

```
True
```

#Q17) Find Missing Numbers

```
num={1,2,3,7,8,10}

full=set(range(1,11))

result=full-num

print(result)
```

Output:

```
{9, 4, 5, 6}
```

#Q18) Remove Duplicates Without set()

```
l=[1,2,2,3,4,4,4]

m=[]

p=[]

for i in l:

    if i not in m:

        p.append(i)
```

```
    m.append(i)
print(p)
```

Output:

```
[1, 2, 3, 4]
```

```
#Q19) Pair with Given Sum
arr=[1,4,5,6,3]
target=7
seen=set()
for n in arr:
    if target-n in seen:
        print("Pair:",n,target-n)
    seen.add(n)
```

Output:

```
Pair: 6 1
Pair: 3 4
```

```
#Q20) Check Disjoint Sets
a={1,2,3,4}
b={5,6,7,8}
if a.isdisjoint(b)==0:
    print("Not a Disjoint sets")
else:
```

```
print("disjoint sets")
```

Output:

disjoint sets
